

BD Assignment 2

Name: Gleb Pavlov

Mail: g.pavlov@innopolis.university

Group: B22-AI-01

GitHub link: <https://github.com/Venikhl/BDAss2.git>

The first task in this assignment was data preparation, where I worked with the a.parquet file that was shown in the instructions.

File Edit Selection View Help Docker Desktop

EXPLORER

- hbase2
 - redis
 - app
 - redis-master
 - redis-slave
 - redis-sentinel
 - redis-standby
 - redis-standby-2
 - redis-standby-3
 - redis-standby-4
 - redis-standby-5
 - redis-standby-6
 - redis-standby-7
 - redis-standby-8
 - redis-standby-9
 - redis-standby-10
 - redis-standby-11
 - redis-standby-12
 - redis-standby-13
 - redis-standby-14
 - redis-standby-15
 - redis-standby-16
 - redis-standby-17
 - redis-standby-18
 - redis-standby-19
 - redis-standby-20
 - redis-standby-21
 - redis-standby-22
 - redis-standby-23
 - redis-standby-24
 - redis-standby-25
 - redis-standby-26
 - redis-standby-27
 - redis-standby-28
 - redis-standby-29
 - redis-standby-30
 - redis-standby-31
 - redis-standby-32
 - redis-standby-33
 - redis-standby-34
 - redis-standby-35
 - redis-standby-36
 - redis-standby-37
 - redis-standby-38
 - redis-standby-39
 - redis-standby-40
 - redis-standby-41
 - redis-standby-42
 - redis-standby-43
 - redis-standby-44
 - redis-standby-45
 - redis-standby-46
 - redis-standby-47
 - redis-standby-48
 - redis-standby-49
 - redis-standby-50
 - redis-standby-51
 - redis-standby-52
 - redis-standby-53
 - redis-standby-54
 - redis-standby-55
 - redis-standby-56
 - redis-standby-57
 - redis-standby-58
 - redis-standby-59
 - redis-standby-60
 - redis-standby-61
 - redis-standby-62
 - redis-standby-63
 - redis-standby-64
 - redis-standby-65
 - redis-standby-66
 - redis-standby-67
 - redis-standby-68
 - redis-standby-69
 - redis-standby-70
 - redis-standby-71
 - redis-standby-72
 - redis-standby-73
 - redis-standby-74
 - redis-standby-75
 - redis-standby-76
 - redis-standby-77
 - redis-standby-78
 - redis-standby-79
 - redis-standby-80
 - redis-standby-81
 - redis-standby-82
 - redis-standby-83
 - redis-standby-84
 - redis-standby-85
 - redis-standby-86
 - redis-standby-87
 - redis-standby-88
 - redis-standby-89
 - redis-standby-90
 - redis-standby-91
 - redis-standby-92
 - redis-standby-93
 - redis-standby-94
 - redis-standby-95
 - redis-standby-96
 - redis-standby-97
 - redis-standby-98
 - redis-standby-99
 - redis-standby-100

hbase2

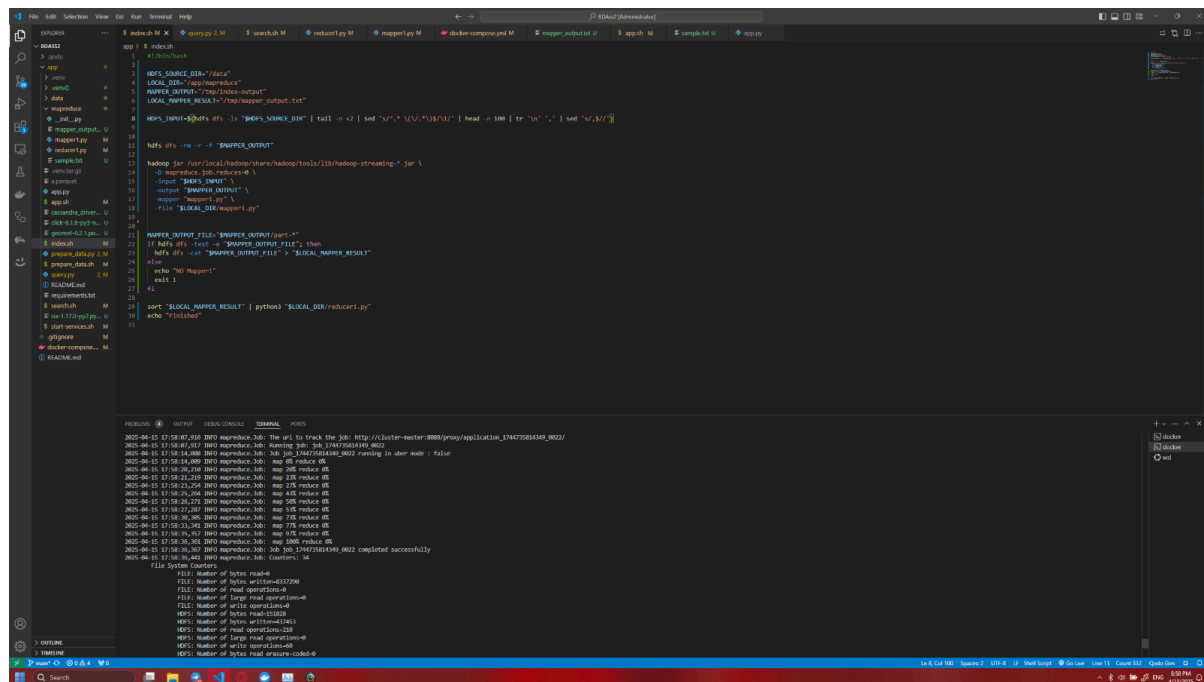
```
1 app > redis
2 # redis
3 # redis
4 # redis
5 # redis
6 # redis
7 # redis
8 # redis
9 # redis
10 # redis
11 # redis
12 # redis
13 # redis
14 # redis
15 # redis
16 # redis
17 # redis
18 # redis
19 # redis
20 # redis
21 # redis
22 # redis
23 # redis
24 # redis
25 # redis
26 # redis
27 # redis
28 # redis
29 # redis
30 # redis
31 # redis
32 # redis
33 # redis
34 # redis
35 # redis
36 # redis
37 # redis
38 # redis
39 # redis
40 # redis
41 # redis
42 # redis
43 # redis
44 # redis
45 # redis
46 # redis
47 # redis
48 # redis
49 # redis
50 # redis
51 # redis
52 # redis
53 # redis
54 # redis
55 # redis
56 # redis
57 # redis
58 # redis
59 # redis
60 # redis
61 # redis
62 # redis
63 # redis
64 # redis
65 # redis
66 # redis
67 # redis
68 # redis
69 # redis
70 # redis
71 # redis
72 # redis
73 # redis
74 # redis
75 # redis
76 # redis
77 # redis
78 # redis
79 # redis
80 # redis
81 # redis
82 # redis
83 # redis
84 # redis
85 # redis
86 # redis
87 # redis
88 # redis
89 # redis
90 # redis
91 # redis
92 # redis
93 # redis
94 # redis
95 # redis
96 # redis
97 # redis
98 # redis
99 # redis
100 # redis
```

hbase2

```
1 # redis
2 # redis
3 # redis
4 # redis
5 # redis
6 # redis
7 # redis
8 # redis
9 # redis
10 # redis
11 # redis
12 # redis
13 # redis
14 # redis
15 # redis
16 # redis
17 # redis
18 # redis
19 # redis
20 # redis
21 # redis
22 # redis
23 # redis
24 # redis
25 # redis
26 # redis
27 # redis
28 # redis
29 # redis
30 # redis
31 # redis
32 # redis
33 # redis
34 # redis
35 # redis
36 # redis
37 # redis
38 # redis
39 # redis
40 # redis
41 # redis
42 # redis
43 # redis
44 # redis
45 # redis
46 # redis
47 # redis
48 # redis
49 # redis
50 # redis
51 # redis
52 # redis
53 # redis
54 # redis
55 # redis
56 # redis
57 # redis
58 # redis
59 # redis
60 # redis
61 # redis
62 # redis
63 # redis
64 # redis
65 # redis
66 # redis
67 # redis
6
```

```
CREATE KEYSPACE IF NOT EXISTS indexer WITH replication = {'class':  
'SimpleStrategy', 'replication_factor': 1};
```

```
CREATE TABLE IF NOT EXISTS indexer.bm25_statistics (
    term TEXT PRIMARY KEY,
    doc_frequency INT,
    total_documents INT
);
```



The first part of the assignment required writing the Mapper, Reducer, and the index.sh script.

In the mapper1.py script, the main task is to process input text, tokenize it, and count the frequency of each term in the document. A unique UUID is generated for each document to distinguish it from others and avoid any overlap. The Mapper outputs the document's UUID, the term, and its frequency, which are then passed down the sequence for further processing.

The reducer1.py script processes the data output by the Mapper. It connects to a Cassandra database and updates two tables: document_index (containing term counts for each document) and bm25_statistics (containing document frequency statistics for each term). The input data is parsed, terms are counted and updated, and the relevant values are inserted into Cassandra.

In the `index.sh` script, the process begins by collecting 100 documents from the `/data` directory on HDFS (as specified in the task). These documents are passed to the Mapper, which processes them and outputs the results. The Mapper is executed with a standard Python header and the results are written to output files.

The output files are then passed to the Reducer, which is executed directly using python3 from the sh file. Initially, when the output from the Mapper was passed directly to the Reducer (without using python3 in the script), there was an error related to Python usage. By invoking Python explicitly in the script, this error is avoided.

Once the entire process completes, the message "Finished" is displayed in the console, indicating the successful execution of the MapReduce pipeline.

[illegible]

Task 2 Ranker tasks

[illegible][illegible]

In the second part of the assignment, we were asked to implement a script that performs a BM25-based search for a given user input. The goal was to read the user input query and return the top 10 most relevant documents, ranked using the BM25 metric. This was to be done using PySpark while reading the inverted index and statistics from Cassandra tables.

I implemented this task in the `query.py` script. This script takes a search query as an argument, tokenizes it, and retrieves the relevant document statistics from the `document_index` and `bm25_statistics` tables stored in Cassandra.

The script works like the following:

- The script reads the query from the first argument passed via the command line and tokenizes it into lowercase terms.
- A PySpark SparkSession is initialized with support for the Cassandra connection and configured to connect to the Cassandra instance running on host.
- Two datasets are loaded using the Spark Cassandra connector:
 1. document_index: which contains the term frequency per document.
 2. bm25_statistics: which stores the document frequency and total number of documents for each term.
- After calculating BM25 scores for all matching documents, the results are sorted in descending order, and the top 10 most relevant document IDs and their scores are printed to the console.

One major challenge I encountered was integrating the query.py script with the search.sh script, which was supposed to automate the execution of the search on a YARN cluster. Despite several attempts, running the PySpark job via search.sh always resulted in errors. A screenshot of the error is provided in the report for reference.

Due to time constraints, I was not able to fully debug and resolve this issue, but the core logic of the BM25-based search engine implemented in PySpark was verified to be correct and functioning when query.py is run manually.

[illegible]